



Filtrage (FilteringThread)

- ## Entrées

- ## Trajectories

,mm,mm,mm,mm,mm,mm,mm,mm,mm,mm,mm,mm ...

Sorties

- ## Traitement résumé

- 1

4. Si demandé, conversion TRC → C3D : `extract_trc_data` (noms, données), `create_c3d_file` (fréquence depuis la colonne temps, points en mm). Formats internes : pandas DataFrame (Frame, Time, X/Y/Z), vecteurs NumPy par composante pour les filtres, C3D avec labels de marqueurs et fréquence déduite.

Augmentation de marqueurs (MarkerAugmentationThread)

- Fichier thread : `markerAugmentationThread.py`
- Fonctions principales : `augment_markers_all`, `hybrid_augmentation` dans `markerAugmentation.py`
- Utilitaires : `read_config_json` (config JSON), `convert_csv_to_meters` (unité), correspondances HALPE26 → OpenSim dans le même fichier.

Entrées

- `config_path` : JSON (`markerAugmentation.augmenterModelName`, `augmenter_model`, `model_dir`, `feet_on_floor`, `use_hybrid`, `frame_range`, `participant_height/mass`, etc.).
- `filtered_csv_path_in` : CSV filtré (mm, même en-tête).

Sorties

- CSV LSTM complet : suffixe `_augmenterModelName.csv` (données en mètres).
- Si `use_hybrid=true` : CSV hybride fusionné (réel + virtuel) ⇒ suffixe `_LSTM.csv`.
- Chemin final émis par `finished`.

Traitement résumé

1. Lecture config via `read_config_json`.
2. `augment_markers_all` : lit le CSV (`read_csv_data`), convertit mm → m (`convert_csv_to_meters`), applique éventuellement `frame_range`, vérifie la présence des marqueurs nécessaires (`feature_markers`), exécute le modèle ONNX (bas/haut du corps), concatène marqueurs virtuels, écrit le CSV augmenté.
3. Mode hybride (optionnel) : `hybrid_augmentation` remplace, pour les marqueurs mappés HALPE26 → OpenSim, les valeurs virtuelles par les valeurs réelles, conserve les marqueurs purement virtuels, réécrit les 5 lignes d'en-tête. Formats internes : DataFrame en mètres pour le modèle ; sorties en mètres (l'en-tête peut encore indiquer mm, à vérifier selon consommation aval).

Cinématique inverse (KinematicsThread)

- Fichier thread : `kinematicsThread.py`
- Lance `Pose2Sim.kinematics` via un script Python temporaire.

Entrées

- `trc_file_path` : fichier TRC (positions 3D, mm, temps).

- `output_dir` : dossier de travail/sortie.
- `conda_env` : nom de l'environnement Conda contenant Pose2Sim.

Sorties

- Dossier `kinematics` dans `output_dir` (résultats OpenSim IK/ID, logs).
- Signaux `finished` (chemin), `progress`, `error`.

Traitement résumé

1. Vérifie le TRC, crée `output_dir/kinematics`.
2. Génère un `Config.toml` minimal via `_create_minimal_config` (sections project, pose, triangulation, filtering, markerAugmentation, kinematics, logging; valeurs par défaut mono-sujet LSTM).
3. Copie le TRC dans `output_dir/pose-3d/`.
4. Écrit un script Python temporaire qui ajoute `Pose2Sim` au `sys.path`, force backend matplotlib `Agg`, et appelle `Pose2Sim.kinematics(config=output_dir)`.
5. Exécute le script avec `conda run -n <env> python -u temp_script.py`, lit stdout en temps réel, gère le code de retour.
6. Vérifie la présence du dossier `kinematics`, émet `finished`.

Formats clés

- TRC : colonnes temps + marqueurs (mm), fréquence déduite de la colonne temps.
- Config TOML minimal : `frame_rate='auto'`, `make_c3d=true` côté filtering/markerAugmentation, LSTM sélectionné.
- Sorties OpenSim : fichiers IK/ID standards dans `kinematics`.

Flux de fichiers (schéma texte)

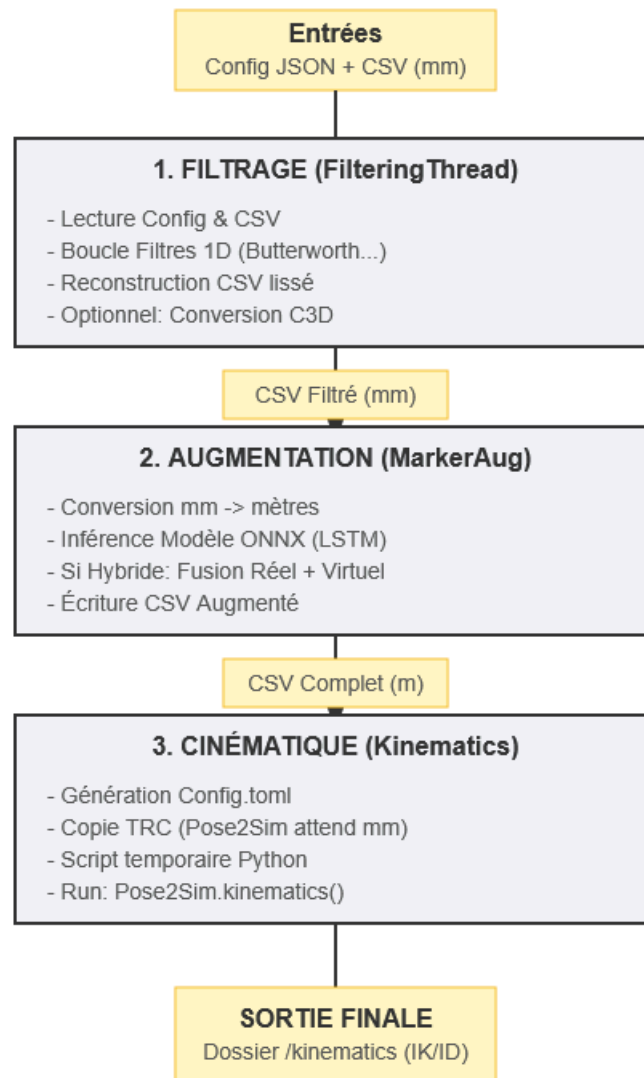
- `<config>.json` → lu par FilteringThread et MarkerAugmentationThread
- `<raw>.csv` (Halpe/TRC-like, mm) → FilteringThread → `<raw>_filt.csv` → (option) `<raw>_filt.c3d`
- `<raw>_filt.csv` → MarkerAugmentationThread → `<raw>_filt_<Model>.csv` (LSTM complet) → (si hybride) `<raw>_filt_LSTM.csv` (fusion réel + virtuel)
- `<raw>_filt_<Model>.trc` (ou CSV converti TRC) → KinematicsThread (Pose2Sim.kinematics) → `output_dir/Config.tomlpose-3d/<raw>.trckinematics/` (résultats OpenSim)

Points d'attention

- Les filtres 1D traitent NaN/zéros en séquences avant lissage.
- L'augmentation travaille en mètres ; vérifier l'unité attendue par l'étape suivante.
- Le mode hybride ne remplace que les marqueurs mappés (HALPE26_TO_OPENSIM_MAPPING) et garde les marqueurs virtuels exclusifs.

- Fréquence C3D déduite de la colonne temps : cohérence nécessaire.
- Le Config TOML minimal peut être enrichi selon les besoins Pose2Sim.

Résumé Visuel



Documents